COS 350 Lecture 11

unix: chmod

unix: umask

stat & lstat

struct stat

subfield coding

user & group ids & names

```
unix> ls -l
total 36
-rwx------ 1 houser houser 13501 Feb 14 17:13 ls2
-rw------- 1 houser houser  3469 Feb 28  2013 ls2.c
-rw------- 1 houser houser  3522 Feb 28  2013 ls2.c~
-rw------- 1 houser houser    19 Mar  4  2013 newfile
-rw-rw-r-- 1 houser houser    33 Feb 28  2013 publicFile
drwx------ 2 houser houser  4096 Feb 28  2013 subdir
```

mode *(permissions)*

number of hard **links**

user name of file owner

group name for file access

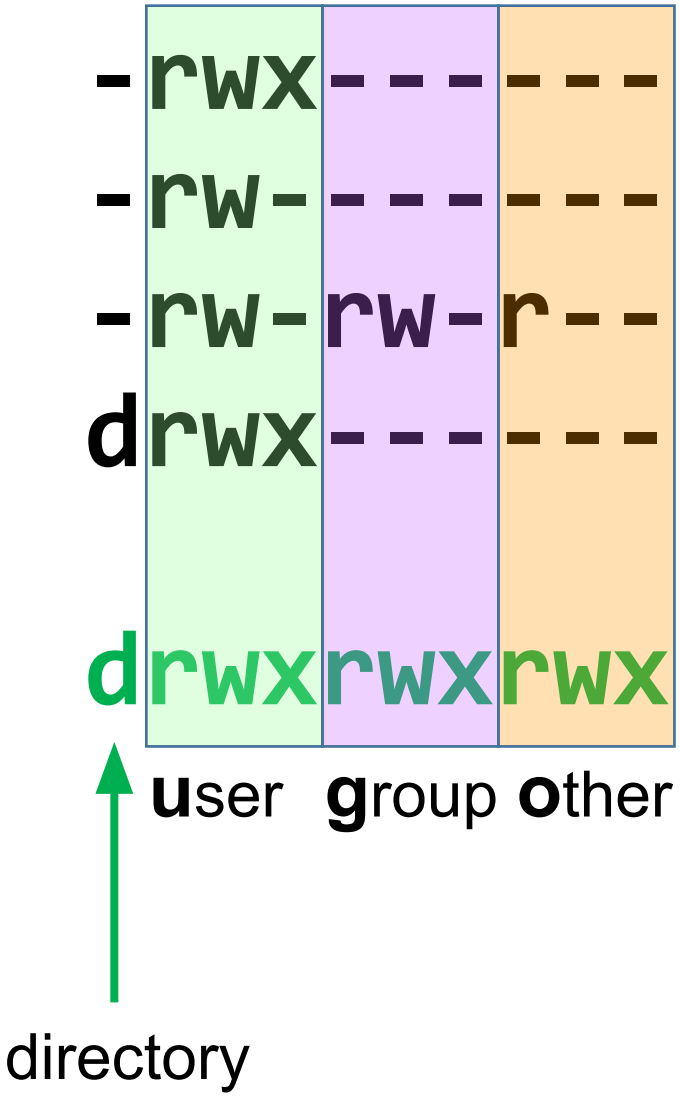
size

time of last **modification**

file name

stored in directory

stored in inode



files:
- r = read
- w = write
- x = execute
  (for programs and scripts)

directories:
- r = read (list) names of files
  in the directory
- w = add, delete, or rename a file
  in the directory
- x = search (access details in inodes
  of files in directory)

Only the "User" (owner) of a file can change its permissions.

symbolic mode:

```
chmod a+r filename
```

who	op	permission
u user	**+** add	**r**
g group	**-** remove	**w**
o other	**=** set	**x**
a all		(can combine)

numeric mode:

```
chmod 640 filename
```

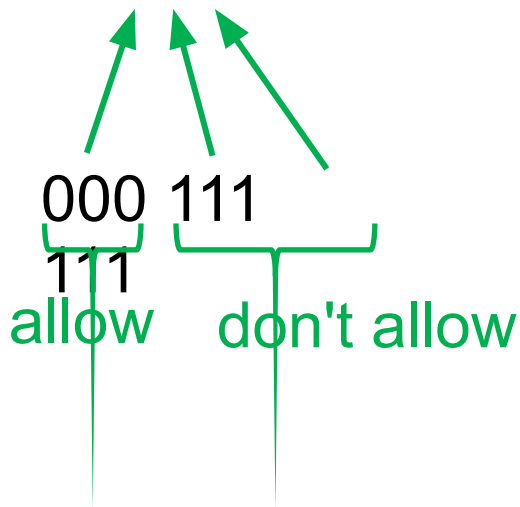
```
rwxrwxrwx
110100000 ←  desired permissions
 6   4   0  ←  encode in octal
```

The default file permission depend on file type:

777 for directories and executables
666 for regular files

These get restricted by **`umask`**
(user's file mode creation mask)

your umask value should be 0077



```
unix> ls -l
total 36
-rwx------ 1 houser houser 13501 Feb 14 17:13 ls2
-rw------- 1 houser houser  3469 Feb 28  2013 ls2.c
-rw------- 1 houser houser  3522 Feb 28  2013 ls2.c~
-rw------- 1 houser houser    19 Mar  4  2013 newfile
-rw-rw-r-- 1 houser houser    33 Feb 28  2013 publicFile
drwx------ 2 houser houser  4096 Feb 28  2013 subdir
```

mode *(permissions)*

number of hard **links**

user name of file owner

group name for file access

size

time of last **modification**

file name

stored in directory

stored in inode

NAME

stat, lstat - get file status

SYNOPSIS

```
#include <sys/stat.h>

int stat(const char *path, struct stat *buf);
int lstat(const char *path, struct stat *buf);
```

DESCRIPTION

These functions return information about a file. No permissions are required on the file itself, but — in the case of stat() and lstat() — execute (search) permission is required on all of the directories in path that lead to the file.

stat() stats the file pointed to by path and fills in buf.

lstat() is identical to stat(), except that if path is a symbolic link, then the link itself is stat-ed, not the file that it refers to.

All of these system calls return a stat structure, which contains the following fields:

```
struct stat {
  dev_t     st_dev;     /* ID of device containing file */
  ino_t     st_ino;     /* inode number */
  mode_t    st_mode;    /* protection */
  nlink_t   st_nlink;   /* number of hard links */
  uid_t     st_uid;     /* user ID of owner */
  gid_t     st_gid;     /* group ID of owner */
  dev_t     st_rdev;    /* device ID (if special file) */
  off_t     st_size;    /* total size, in bytes */
  blksize_t st_blksize; /* blocksize for file system I/O */
  blkcnt_t  st_blocks;  /* number of 512B blocks allocated
  time_t    st_atime;   /* time of last access */
  time_t    st_mtime;   /* time of last modification */
  time_t    st_ctime;   /* time of last status change */
};
```

book's case study: `ls2.c` does `ls -l`

```
main(int ac, char *av[])
```

```
void do_ls( char dirname[] )
```

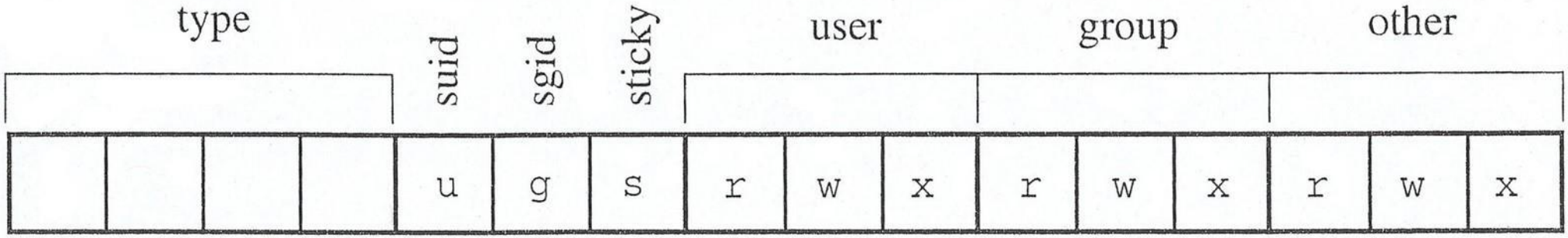
```
void dostat( char *filename )
```

```
 void show_file_info( char *filename, struct stat *info_p )
{
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

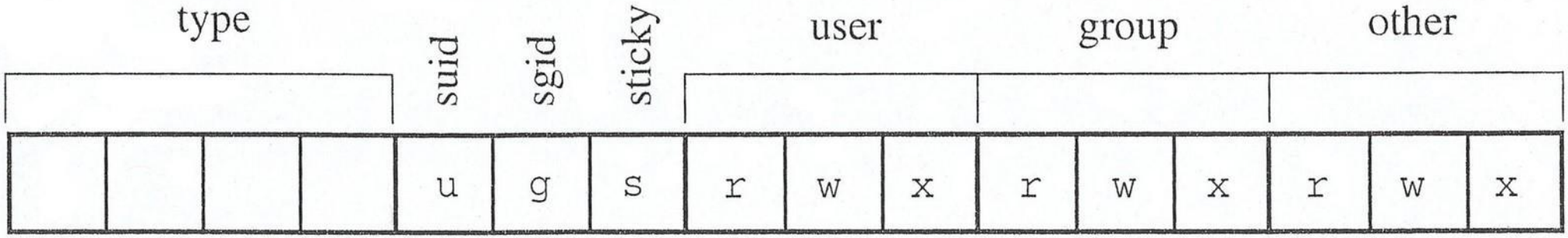
```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

type				suid	sgid	sticky	user			group			other		
				u	g	s	r	w	x	r	w	x	r	w	x

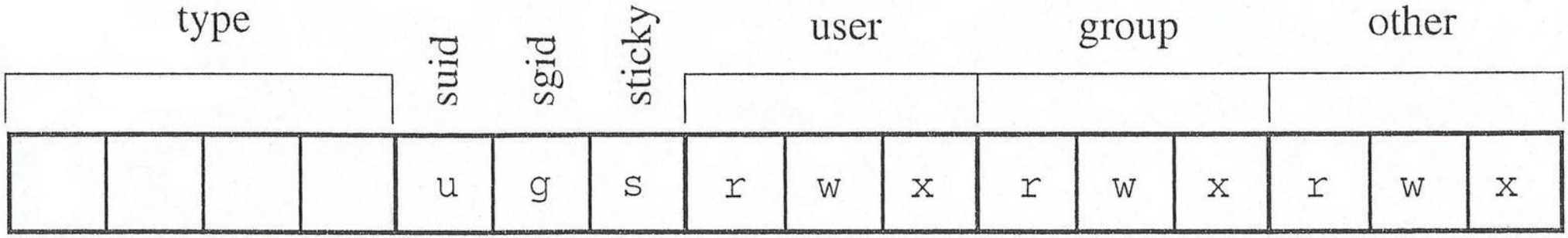


excerpts from sys/stat.h

```
/* Protection bits.  */
#define S_IRWXU   00700   /* read, write, execute: owner */
#define S_IRUSR   00400   /* read permission: owner */
#define S_IWUSR   00200   /* write permission: owner */
#define S_IXUSR   00100   /* execute permission: owner */
#define S_IRWXG   00070   /* read, write, execute: group */
#define S_IRGRP   00040   /* read permission: group */
#define S_IWGRP   00020   /* write permission: group */
#define S_IXGRP   00010   /* execute permission: group */
#define S_IRWXO   00007   /* read, write, execute: other */
#define S_IROTH   00004   /* read permission: other */
#define S_IWOTH   00002   /* write permission: other */
#define S_IXOTH   00001   /* execute permission: other */
```




```
excerpts from sys/stat.h

/* Encoding of the file mode.  */
#define S_IFMT   0170000 // These bits determine file type

/* File types.  */
#define S_IFREG  0100000          /* Regular file.  */
#define S_IFDIR  0040000          /* Directory.  */
#define S_IFCHR  0020000          /* Character device.  */
#define S_IFBLK  0060000          /* Block device.  */
#define S_IFIFO  0010000          /* FIFO.  */
#define S_IFLNK  0120000          /* Symbolic link.  */
#define S_IFSOCK 0140000          /* Socket.  */
```




```
/* File type macros */
#define S_ISREG(mode)    (((mode)&0170000) == 0100000)
#define S_ISDIR(mode)    (((mode)&0170000) == 0040000)
#define S_ISCHR(mode)    (((mode)&0170000) == 0020000)
#define S_ISBLK(mode)    (((mode)&0170000) == 0060000)
#define S_ISFIFO(mode)   (((mode)&0170000) == 0010000)
#define S_ISLNK(mode)    (((mode)&0170000) == 0120000)
#define S_ISSOCK(mode)   (((mode)&0170000) == 0140000)
```

```
// This function takes a mode value and a char array and
// puts into the char array the file type and the nine
// letters that correspond to the bits in mode.
// NOTE: It does not code setuid, setgid, and sticky codes
void mode_to_letters( int mode, char str[] )
{
  strcpy( str, "----------" );        /* default=no perms */

  if ( S_ISDIR(mode) )  str[0] = 'd'; /* directory?       */
  if ( S_ISCHR(mode) )  str[0] = 'c'; /* char devices     */
  if ( S_ISBLK(mode) )  str[0] = 'b'; /* block device     */

  if ( mode & S_IRUSR ) str[1] = 'r'; /* 3 bits for user  */
  if ( mode & S_IWUSR ) str[2] = 'w';
  if ( mode & S_IXUSR ) str[3] = 'x';

  if ( mode & S_IRGRP ) str[4] = 'r'; /* 3 bits for group */
  if ( mode & S_IWGRP ) str[5] = 'w';
  if ( mode & S_IXGRP ) str[6] = 'x';

  if ( mode & S_IROTH ) str[7] = 'r'; /* 3 bits for other */
  if ( mode & S_IWOTH ) str[8] = 'w';
  if ( mode & S_IXOTH ) str[9] = 'x';
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

excerpts from the password file

username
password
uid
gid
real name
home directory
shell

```
alagic:x:20340:20340:Suad Alagic:/home/faculty/alagic:/bin/bash
boothe:x:20266:20266:Robert Boothe:/home/faculty/boothe:/bin/bash
briggs:x:20263:20263:David Briggs:/home/faculty/briggs:/bin/bash
congdon:x:20966:20966:Clare Congdon:/home/faculty/congdon:/bin/bash
houser:x:20287:20287:Stephen Houser:/home/faculty/houser:/bin/bash
macleod:x:20262:20262:Bruce Macleod:/home/faculty/macleod:/bin/bash
welty:x:20264:20264:Charles Welty:/home/faculty/welty:/bin/bash
. . .
root:x:0:0:root:/root:/bin/bash
. . .
```

NAME

getpwnam, **getpwuid** - get password file entry

SYNOPSIS

```
#include <pwd.h>

struct passwd *getpwnam(const char *name);

struct passwd *getpwuid(uid_t uid);
```

DESCRIPTION

The **getpwnam()** function returns a pointer to a structure containing the record in the password database (e.g., the local password file /etc/passwd, NIS, and LDAP) that matches the **username**.

The **getpwuid()** function returns a pointer to a structure containing the record in the password database that matches the user ID **uid**.

The passwd structure is defined in <pwd.h> as follows:

```
struct passwd {
    char   *pw_name;        /* username */
    char   *pw_passwd;      /* user password */
    uid_t   pw_uid;         /* user ID */
    gid_t   pw_gid;         /* group ID */
    char   *pw_gecos;       /* real name */
    char   *pw_dir;         /* home directory */
    char   *pw_shell;       /* shell program */
};
```

```c
#include <pwd.h>

/*
 * returns pointer to username associated with uid,
 * uses getpwuid()
 */
char *uid_to_name( uid_t uid )
{
  struct passwd *pw_ptr;
  static char numstr[10];

  if ( ( pw_ptr = getpwuid( uid ) ) == NULL ){
    sprintf(numstr,"%d", uid);
    return numstr;
  }
  else
    return pw_ptr->pw_name ;
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```

NAME

ctime

SYNOPSIS

```
#include <time.h>

char *ctime(const time_t *timep);
```

DESCRIPTION

The ctime() functions takes an argument of data type time_t which represents calendar time. When interpreted as an absolute time value, it represents the number of seconds elapsed since 00:00:00 on January 1, 1970, Coordinated Universal Time (UTC).

The call ctime(t) converts the calendar time t into a null-terminated string of the form

"Wed Jun 30 21:49:08 1993\n"

NAME

ctime

SYNOPSIS

```
#include <time.h>

char *ctime(const time_t *timep);
```

DESCRIPTION

The ctime() functions takes an argument of data type time_t which represents calendar time. When interpreted as an absolute time value, it represents the number of seconds elapsed since 00:00:00 on January 1, 1970, Coordinated Universal Time (UTC).

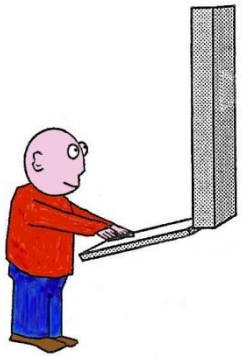
The call ctime(t) converts the calendar time t into a null-terminated string of the form

"Wed Jun 30 21:49:08 1993\n"

```
printf( "%.12s ", 4+ctime(&info_p->st_mtime));
```

filled in stat structure

```
/*
 * display the info about 'filename'.
 * The info is stored in struct at *info_p
 */
void show_file_info( char *filename, struct stat *info_p )
{
  char *uid_to_name(), *gid_to_name(), *filemode();
  void mode_to_letters();
  char modestr[11];

  mode_to_letters( info_p->st_mode, modestr );

  printf( "%s"    , modestr );
  printf( "%4d "  , (int) info_p->st_nlink);
  printf( "%-8s " , uid_to_name(info_p->st_uid) );
  printf( "%-8s " , gid_to_name(info_p->st_gid) );
  printf( "%8ld " , (long)info_p->st_size);
  printf( "%.12s ", 4+ctime(&info_p->st_mtime));
  printf( "%s\n"  , filename );
}
```



ls2.c